PFO 3: Rediseño como Sistema Distribuido (Cliente-Servidor)

Programación sobre redes · Tecnicatura ITFS 29 · Trabajo práctico

Alumno: Adriano Caloni
Repositorio: https://github.com/AdrianoJere/pfo3-sistema-distribuido-sockets

1. Objetivo

Rediseñar el sistema cliente-servidor desarrollado en el PFO1 como un **sistema distribuido**: los clientes envían tareas por socket a un servidor que las reparte entre varios procesos *worker* independientes, cada uno con su propio pool de hilos. Los resultados se guardan en un almacenamiento compartido y vuelven al cliente que los pidió.

2. Diagrama del sistema

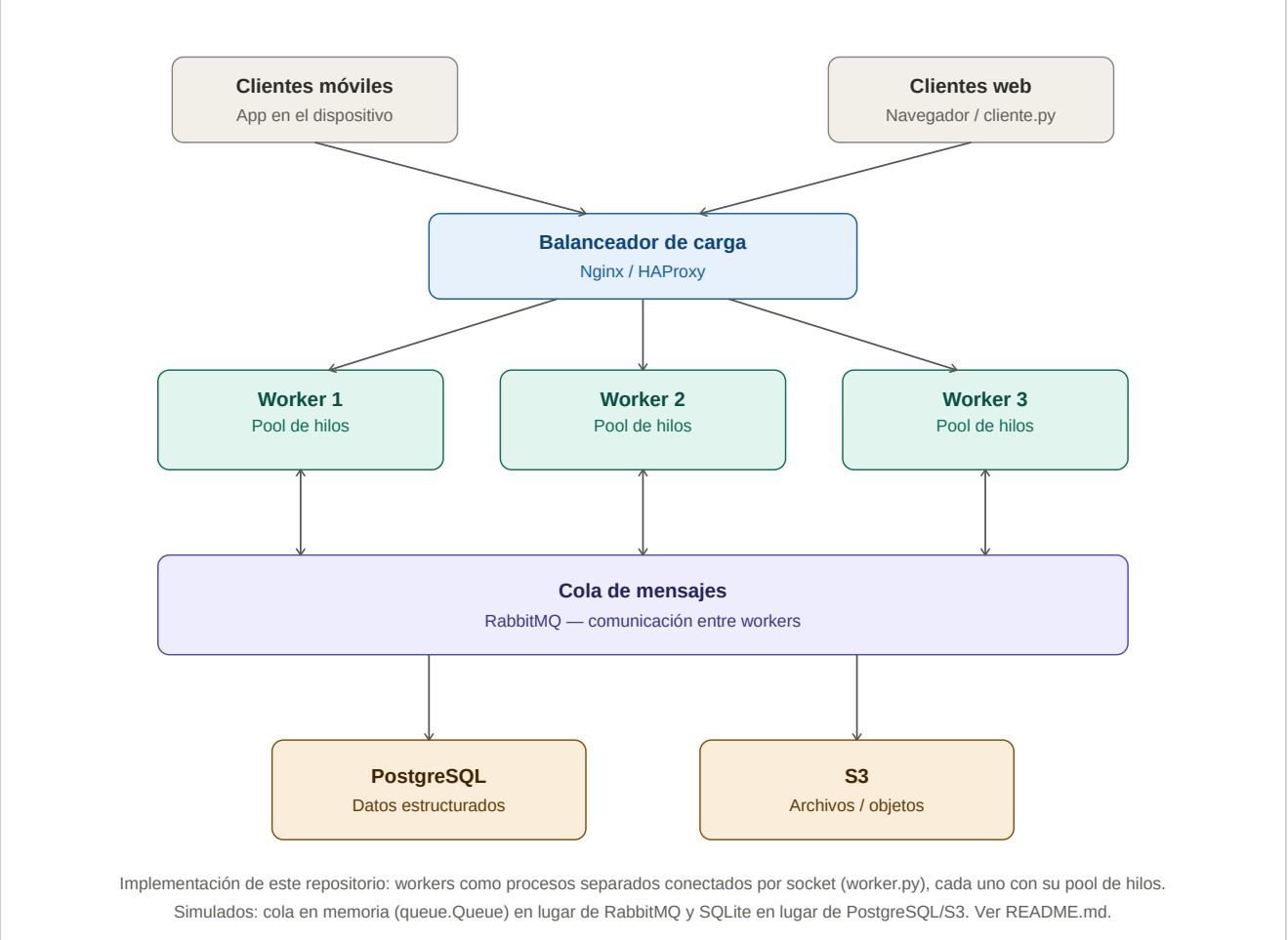


Figura 1. Arquitectura distribuida: clientes → balanceador de carga → workers (pool de hilos) ↔ cola de mensajes → almacenamiento distribuido.

Componentes

- **Clientes (móvil / web):** envían tareas al sistema.

- **Balanceador de carga (Nginx / HAProxy):** recibe las conexiones y reparte el trabajo entre los servidores.
- **Servidores worker:** cada uno procesa tareas en paralelo con un pool de hilos; se pueden agregar más para escalar horizontalmente.
- **Cola de mensajes (RabbitMQ):** desacopla la recepción de tareas de su procesamiento y permite la comunicación entre servidores.
- **Almacenamiento distribuido (PostgreSQL, S3):** guarda los resultados de forma compartida y persistente.

3. Implementación en Python

Los workers son **procesos independientes** que se comunican con el servidor por socket, como en una arquitectura distribuida real. Las herramientas de infraestructura se **simulan** con la librería estándar de Python, para poder ejecutar todo sin instalar servicios externos:

Componente del diagrama	Herramienta real	Implementación en este repositorio
Clientes	App móvil / navegador	`cliente.py`: envía tareas por socket (puerto 5000)
Balanceador de carga	Nginx / HAProxy	`servidor.py`: recibe las conexiones y reparte las tareas entre los workers
Servidores worker	Varias máquinas con pool de hilos	`worker.py`: un proceso por worker con `ThreadPoolExecutor`; se conecta por socket (puerto 5001)
Cola de mensajes	RabbitMQ	`queue.Queue()` en memoria dentro de `servidor.py`
Almacenamiento distribuido	PostgreSQL / S3	SQLite (`resultados.db`) con un `Lock` para escrituras concurrentes

Cómo funciona

- Los **clientes** hablan con el servidor por el puerto 5000 y envían un mensaje JSON por conexión: `{"operacion": "contar_palabras", "texto": "hola mundo"}`.
- Los **workers** se conectan al puerto 5001 y se registran con su nombre y su cantidad de hilos.
- El servidor deja cada tarea en la cola y la entrega al primer worker con capacidad libre. Cada worker acepta como máximo tantas tareas simultáneas como hilos tiene (semáforo), por lo que la carga se reparte en proporción a la capacidad de cada uno.
- Cuando un worker termina, devuelve el resultado al servidor, que lo guarda en la base y se lo responde al cliente indicando qué worker e hilo lo procesó.
- Si un worker se desconecta con tareas en curso, el servidor las **vuelve a encolar** y las toma otro worker (tolerancia a fallos).
- Si no hay ningún worker conectado, el servidor responde con un error en lugar de dejar al cliente esperando.

Operaciones disponibles: contar palabras, invertir texto y calcular hash SHA-256. Cada worker simula 0,5 s de trabajo por tarea para que el reparto sea visible.

Cómo ejecutarlo

Solo requiere Python 3 (sin dependencias externas). En terminales separadas, dentro de la carpeta del proyecto:

```
python servidor.py
python worker.py Worker-A 2
python worker.py Worker-B 4
python cliente.py
python prueba_carga.py 12     # opcional: 12 tareas en paralelo para ver el reparto
```

4. Pruebas realizadas

4.1 Reparto de carga entre workers

Con el servidor, Worker-A (2 hilos), Worker-B (4 hilos) y `prueba_carga.py 12`, las 12 tareas simultáneas se repartieron 4 y 8, proporcional a los hilos de cada worker, y terminaron en 1,1 s (en serie habrían tardado 6 s).

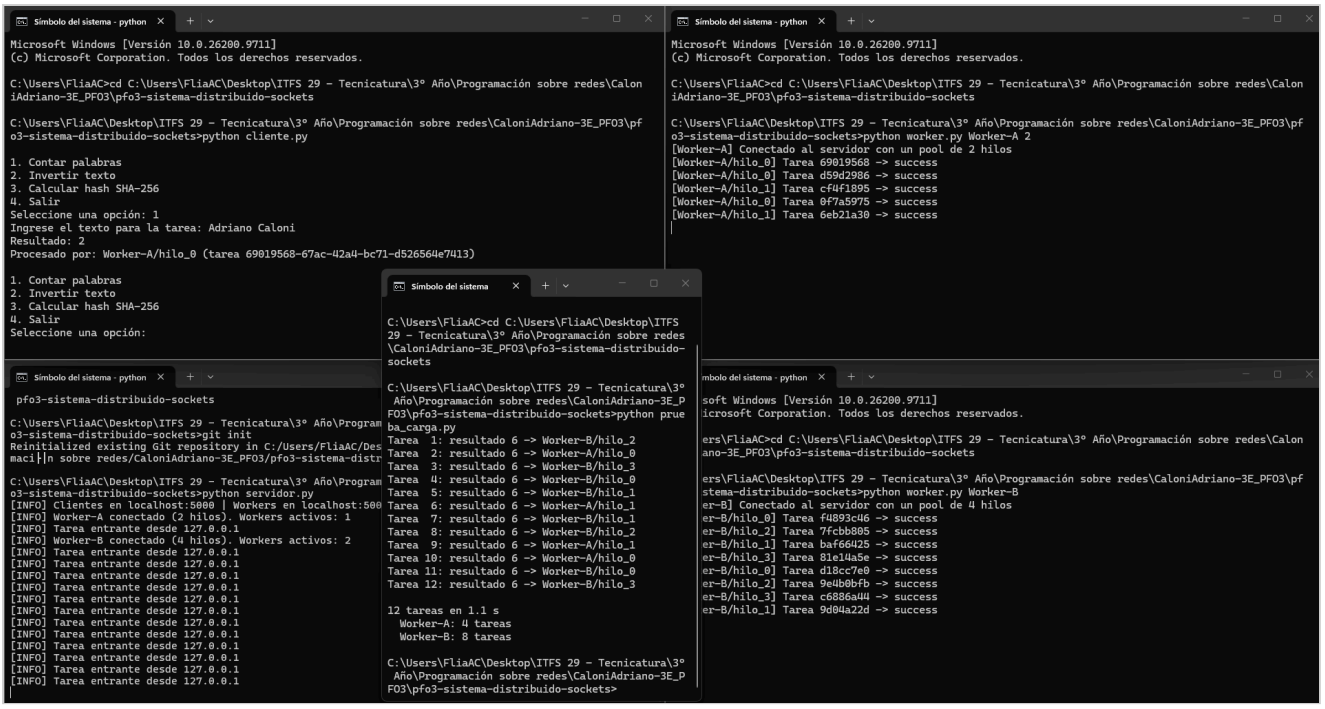


Figura 2. Servidor, workers y salida de la prueba de carga: 12 tareas en 1,1 s (Worker-A: 4, Worker-B: 8).

4.2 Tolerancia a fallos

Con la demora simulada elevada a 5 s, se lanzaron 12 tareas y se cerró Worker-A en pleno procesamiento. El servidor detectó la desconexión y re-encoló las 2 tareas que tenía en curso (`Tareas re-encoladas: 2`); Worker-B las completó junto con el resto.

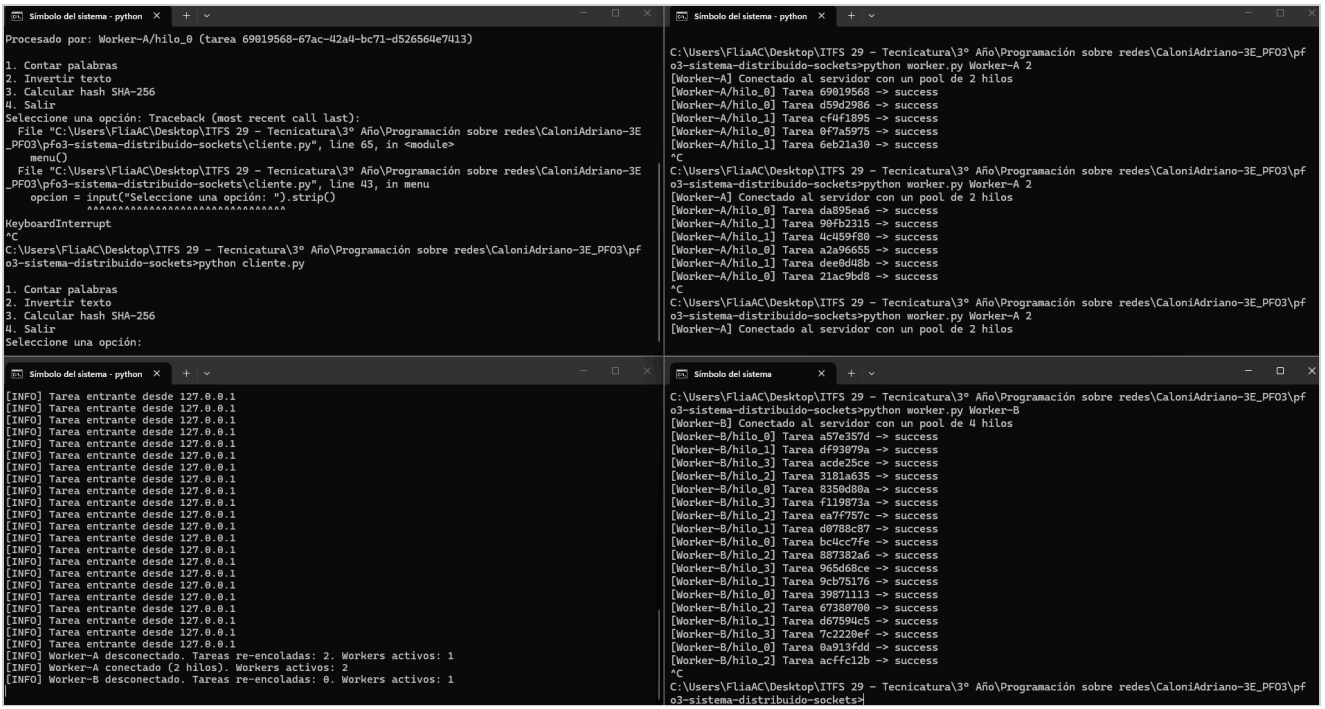


Figura 3. Servidor con el aviso «Worker-A desconectado. Tareas re-encoladas: 2» y Worker-B procesando el resto.

4.3 Otras verificaciones

- Las tres operaciones devuelven el resultado correcto.
- Con ningún worker conectado, el cliente recibe «No hay workers disponibles».
- Una operación inexistente o un mensaje que no es JSON devuelven un error controlado sin afectar al servidor.
- Todos los resultados quedan registrados en `resultados.db`, con el worker y el hilo que los procesaron.

5. Limitaciones conocidas

- La cola de mensajes es en memoria: si el servidor se reinicia, las tareas pendientes se pierden (RabbitMQ real las persiste en disco).
- El servidor es un único punto de entrada: el balanceador real (Nginx/HAProxy) repartiría conexiones entre varias instancias de servidor.
- Los workers de esta demo corren en la misma máquina (`localhost`); para usar máquinas distintas hay que cambiar el host en `servidor.py` (`0.0.0.0`) y en `worker.py` (dirección del servidor).

6. Código fuente

servidor.py

```
"""Servidor distribuido: recibe tareas de los clientes por socket (puerto 5000),
las pone en una cola (simula RabbitMQ) y las despacha por socket a procesos
worker independientes (puerto 5001, ver worker.py). Cuando un worker devuelve
el resultado, se guarda en SQLite (simula PostgreSQL/S3) y se responde al cliente."""
import json
import queue
import socket
import sqlite3
import sys
import threading
import uuid
from datetime import datetime

HOST = "localhost"
PORT_CLIENTES = 5000
PORT_WORKERS = 5001
DB_NAME = "resultados.db"

# Cola de tareas pendientes de despacho: simula la cola de RabbitMQ.
cola_tareas = queue.Queue()

# tarea_id -> conexión del cliente que está esperando el resultado
pendientes = {}
lock_pendientes = threading.Lock()

# nombre -> info del worker conectado (para saber si hay alguno disponible)
workers = {}
lock_workers = threading.Lock()

# Lock para que dos hilos no escriban en la DB al mismo tiempo (sección crítica)
lock_db = threading.Lock()


def inicializar_db():
    """Crea el almacenamiento distribuido (simulado con SQLite) si no existe."""
    try:
        conexion = sqlite3.connect(DB_NAME)
        conexion.execute(
            """CREATE TABLE IF NOT EXISTS resultados (
                   id TEXT PRIMARY KEY,
                   operacion TEXT NOT NULL,
                   entrada TEXT NOT NULL,
                   resultado TEXT NOT NULL,
                   procesado_por TEXT NOT NULL,
                   fecha TEXT NOT NULL
               )"""
        )
        conexion.commit()
        conexion.close()
    except sqlite3.Error as e:
        print(f"[ERROR] No se pudo acceder al almacenamiento: {e}")
        sys.exit(1)


def guardar_resultado(tarea, resultado, procesado_por):
    """Persiste el resultado en el almacenamiento distribuido (simulado)."""
    fecha = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with lock_db:
        conexion = sqlite3.connect(DB_NAME)
        conexion.execute(
            "INSERT INTO resultados (id, operacion, entrada, resultado, procesado_por, fecha) "
            "VALUES (?, ?, ?, ?, ?, ?)",
            (tarea["id"], tarea["operacion"], tarea["texto"], resultado, procesado_por, fecha),
        )
        conexion.commit()
        conexion.close()


def crear_socket_servidor(puerto):
```

```
    """Crea y configura un socket de escucha en el puerto indicado."""
    servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # En Windows no se activa SO_REUSEADDR: ahí permite abrir dos servidores
    # en el mismo puerto y no se detectaría el error de "puerto ocupado".
    if sys.platform != "win32":
        servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    try:
        servidor.bind((HOST, puerto))
    except OSError as e:
        print(f"[ERROR] No se pudo usar el puerto {puerto} (¿ocupado?): {e}")
        sys.exit(1)
    servidor.listen(50)
    servidor.settimeout(1.0)  # permite que Ctrl+C funcione también en Windows
    return servidor


def responder_cliente(conexion, respuesta):
    """Envía la respuesta JSON al cliente y cierra su conexión."""
    try:
        conexion.sendall(json.dumps(respuesta).encode("utf-8"))
    except (ConnectionError, OSError):
        print("[AVISO] El cliente se desconectó antes de recibir la respuesta")
    finally:
        conexion.close()


def finalizar_tarea(mensaje, nombre_worker):
    """Procesa el resultado devuelto por un worker y responde al cliente."""
    with lock_pendientes:
        entrada = pendientes.pop(mensaje.get("tarea_id"), None)
    if entrada is None:
        return  # tarea desconocida o ya respondida
    tarea, conexion_cliente = entrada

    if mensaje.get("status") == "success":
        procesado_por = mensaje.get("procesado_por", nombre_worker)
        try:
            guardar_resultado(tarea, mensaje["resultado"], procesado_por)
            respuesta = {
                "status": "success",
                "tarea_id": tarea["id"],
                "resultado": mensaje["resultado"],
                "procesado_por": procesado_por,
            }
        except sqlite3.Error as e:
            respuesta = {"status": "error", "message": f"No se pudo guardar el resultado: {e}"}
    else:
        respuesta = {"status": "error", "message": mensaje.get("message", "Error en el worker")}
    responder_cliente(conexion_cliente, respuesta)


def enviar_tareas_al_worker(conexion, nombre, semaforo, estado):
    """Hilo emisor: saca tareas de la cola y las manda al worker por socket.
    El semáforo limita las tareas en vuelo a la cantidad de hilos del worker,
    así la carga se reparte entre workers en vez de acapararla uno solo."""
    while not estado["cerrado"]:
        if not semaforo.acquire(timeout=1):
            continue
        try:
            tarea = cola_tareas.get(timeout=1)
        except queue.Empty:
            semaforo.release()
            continue

        with estado["lock"]:
            if estado["cerrado"]:  # el worker se cayó mientras esperábamos
                cola_tareas.put(tarea)
                return
            estado["en_vuelo"][tarea["id"]] = tarea
        try:
            conexion.sendall((json.dumps(tarea) + "\n").encode("utf-8"))
        except OSError:
            # Se despierta al hilo receptor: él re-encola lo que quedó en vuelo.
            try:
                conexion.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass
            return
```

```
def atender_worker(conexion, direccion):
    """Registra un worker y recibe sus resultados. Si el worker se desconecta,
    las tareas que tenía en vuelo vuelven a la cola para otro worker."""
    lector = conexion.makefile("r", encoding="utf-8")
    try:
        registro = json.loads(lector.readline())
        nombre = str(registro["nombre"])
        capacidad = max(1, int(registro.get("hilos", 1)))
    except (ValueError, KeyError, TypeError):
        print(f"[AVISO] Registro de worker inválido desde {direccion[0]}")
        conexion.close()
        return

    estado = {"cerrado": False, "en_vuelo": {}, "lock": threading.Lock()}
    semaforo = threading.Semaphore(capacidad)
    with lock_workers:
        workers[nombre] = capacidad
    print(f"[INFO] {nombre} conectado ({capacidad} hilos). Workers activos: {len(workers)}")

    threading.Thread(
        target=enviar_tareas_al_worker, args=(conexion, nombre, semaforo, estado), daemon=True
    ).start()

    try:
        for linea in lector:
            try:
                mensaje = json.loads(linea)
            except json.JSONDecodeError:
                continue
            with estado["lock"]:
                estado["en_vuelo"].pop(mensaje.get("tarea_id"), None)
            semaforo.release()
            finalizar_tarea(mensaje, nombre)
    except (ConnectionError, OSError):
        pass
    finally:
        with estado["lock"]:
            estado["cerrado"] = True
            huerfanas = list(estado["en_vuelo"].values())
            estado["en_vuelo"].clear()
        for tarea in huerfanas:
            cola_tareas.put(tarea)
        with lock_workers:
            workers.pop(nombre, None)
        conexion.close()
        print(f"[INFO] {nombre} desconectado. Tareas re-encoladas: {len(huerfanas)}. "
              f"Workers activos: {len(workers)}")


def aceptar_workers(servidor_workers):
    """Hilo que acepta las conexiones de los procesos worker."""
    while True:
        try:
            conexion, direccion = servidor_workers.accept()
        except socket.timeout:
            continue
        except OSError:
            return
        conexion.settimeout(None)
        threading.Thread(target=atender_worker, args=(conexion, direccion), daemon=True).start()


def recibir_tarea(conexion):
    """Lee y parsea el JSON enviado por el cliente. Devuelve None si es inválido."""
    try:
        datos = conexion.recv(4096)
        return json.loads(datos.decode("utf-8")) if datos else None
    except (json.JSONDecodeError, UnicodeDecodeError, OSError):
        return None


def aceptar_clientes(servidor_clientes):
    """Acepta clientes y encola cada tarea recibida (rol de dispatcher/balanceador)."""
    while True:
        try:
            conexion, direccion = servidor_clientes.accept()
```

```
        except socket.timeout:
            continue
        print(f"[INFO] Tarea entrante desde {direccion[0]}")

        peticion = recibir_tarea(conexion)
        if not isinstance(peticion, dict) or "operacion" not in peticion or "texto" not in peticion:
            responder_cliente(
                conexion, {"status": "error", "message": "JSON inválido. Se espera {operacion, texto}"}
            )
            continue

        with lock_workers:
            hay_workers = bool(workers)
        if not hay_workers:
            responder_cliente(
                conexion, {"status": "error", "message": "No hay workers disponibles. Iniciá worker.py"}
            )
            continue

        tarea = {
            "id": str(uuid.uuid4()),
            "operacion": peticion["operacion"],
            "texto": peticion["texto"],
        }
        with lock_pendientes:
            pendientes[tarea["id"]] = (tarea, conexion)
        cola_tareas.put(tarea)  # equivale a publicar un mensaje en RabbitMQ


def main():
    inicializar_db()
    servidor_clientes = crear_socket_servidor(PORT_CLIENTES)
    servidor_workers = crear_socket_servidor(PORT_WORKERS)
    print(f"[INFO] Clientes en {HOST}:{PORT_CLIENTES} | Workers en {HOST}:{PORT_WORKERS}")
    threading.Thread(target=aceptar_workers, args=(servidor_workers,), daemon=True).start()
    try:
        aceptar_clientes(servidor_clientes)
    except KeyboardInterrupt:
        print("\n[INFO] Servidor detenido")
    finally:
        servidor_clientes.close()
        servidor_workers.close()


if __name__ == "__main__":
    main()
```

worker.py

```
"""Worker: proceso independiente que se conecta al servidor por socket, recibe
tareas y las procesa con su propio pool de hilos. Se pueden iniciar varios
workers en terminales distintas:

    python worker.py Worker-A
    python worker.py Worker-B 2     (nombre y cantidad de hilos)
"""
import hashlib
import json
import socket
import sys
import threading
import time
from concurrent.futures import ThreadPoolExecutor

HOST = "localhost"
PORT_WORKERS = 5001
HILOS_POR_DEFECTO = 4
DEMORA_SIMULADA = 0.5  # segundos: simula que cada tarea tarda, para ver el reparto

# Varios hilos del pool responden por el mismo socket: hace falta un lock de envío
lock_envio = threading.Lock()


def procesar_tarea(operacion, texto):
    """Ejecuta la operación solicitada. Lanza ValueError si la operación no existe."""
    time.sleep(DEMORA_SIMULADA)
```

```
    if operacion == "contar_palabras":
        return str(len(texto.split()))
    elif operacion == "invertir_texto":
        return texto[::-1]
    elif operacion == "hash_sha256":
        return hashlib.sha256(texto.encode("utf-8")).hexdigest()
    else:
        raise ValueError(f"Operación no soportada: {operacion}")


def enviar(conexion, mensaje):
    """Envía un mensaje JSON (una línea) al servidor, de forma segura entre hilos."""
    with lock_envio:
        conexion.sendall((json.dumps(mensaje) + "\n").encode("utf-8"))


def atender_tarea(conexion, nombre, tarea):
    """Tarea del pool de hilos: procesa y devuelve el resultado al servidor."""
    hilo = threading.current_thread().name
    try:
        resultado = procesar_tarea(tarea["operacion"], tarea["texto"])
        mensaje = {
            "tarea_id": tarea["id"],
            "status": "success",
            "resultado": resultado,
            "procesado_por": f"{nombre}/{hilo}",
        }
    except (KeyError, ValueError) as e:
        mensaje = {"tarea_id": tarea.get("id"), "status": "error", "message": str(e)}
    print(f"[{nombre}/{hilo}] Tarea {str(tarea.get('id'))[:8]} -> {mensaje['status']}")
    try:
        enviar(conexion, mensaje)
    except OSError:
        print(f"[{nombre}] No se pudo enviar el resultado: servidor desconectado")


def main():
    nombre = sys.argv[1] if len(sys.argv) > 1 else "Worker-1"
    try:
        hilos = int(sys.argv[2]) if len(sys.argv) > 2 else HILOS_POR_DEFECTO
    except ValueError:
        print("La cantidad de hilos debe ser un número entero")
        return

    conexion = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        conexion.connect((HOST, PORT_WORKERS))
    except ConnectionRefusedError:
        print("No se pudo conectar: ¿está el servidor en ejecución?")
        return

    enviar(conexion, {"nombre": nombre, "hilos": hilos})
    print(f"[{nombre}] Conectado al servidor con un pool de {hilos} hilos")

    lector = conexion.makefile("r", encoding="utf-8")
    # El nombre de cada hilo (hilo_0, hilo_1...) aparece en el resultado: Worker-A/hilo_0
    with ThreadPoolExecutor(max_workers=hilos, thread_name_prefix="hilo") as pool:
        try:
            for linea in lector:
                try:
                    tarea = json.loads(linea)
                except json.JSONDecodeError:
                    continue
                pool.submit(atender_tarea, conexion, nombre, tarea)
        except KeyboardInterrupt:
            print(f"\n[{nombre}] Worker detenido")
        except (ConnectionError, OSError):
            pass
    print(f"[{nombre}] Desconectado del servidor")
    conexion.close()


if __name__ == "__main__":
    main()
```

cliente.py

```
"""Cliente: envía tareas al servidor distribuido (simulado) y muestra el
resultado procesado por el worker que la haya tomado."""
import json
import socket

HOST = "localhost"
PORT = 5000

OPERACIONES = {
    "1": "contar_palabras",
    "2": "invertir_texto",
    "3": "hash_sha256",
}


def enviar_tarea(operacion, texto):
    """Abre una conexión nueva, envía la tarea en JSON y devuelve la respuesta."""
    cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        cliente.connect((HOST, PORT))
    except ConnectionRefusedError:
        print("No se pudo conectar: ¿está el servidor en ejecución?")
        return None

    try:
        peticion = {"operacion": operacion, "texto": texto}
        cliente.sendall(json.dumps(peticion).encode("utf-8"))
        datos = cliente.recv(4096)
        if not datos:
            print("El servidor cerró la conexión sin responder.")
            return None
        return json.loads(datos.decode("utf-8"))
    except (ConnectionError, OSError) as e:
        print(f"Error de conexión: {e}")
        return None
    finally:
        cliente.close()


def menu():
    while True:
        print("\n1. Contar palabras\n2. Invertir texto\n3. Calcular hash SHA-256\n4. Salir")
        opcion = input("Seleccione una opción: ").strip()

        if opcion == "4":
            print("Saliendo...")
            break
        if opcion not in OPERACIONES:
            print("Opción inválida.")
            continue

        texto = input("Ingrese el texto para la tarea: ")
        respuesta = enviar_tarea(OPERACIONES[opcion], texto)

        if respuesta is None:
            continue
        if respuesta.get("status") == "success":
            print(f"Resultado: {respuesta['resultado']}")
            print(f"Procesado por: {respuesta['procesado_por']} (tarea {respuesta['tarea_id']})")
        else:
            print(f"Error: {respuesta.get('message')}")


if __name__ == "__main__":
    menu()
```

prueba_carga.py

```
"""Prueba de carga: envía varias tareas en paralelo (una conexión por tarea) y
muestra qué worker e hilo procesó cada una. Sirve para ver el reparto entre
workers.  Uso:  python prueba_carga.py [cantidad_de_tareas]"""
import json
import socket
```

```
import sys
import threading
import time

HOST = "localhost"
PORT = 5000
resultados = {}


def enviar_tarea(numero):
    cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        cliente.connect((HOST, PORT))
        peticion = {"operacion": "contar_palabras", "texto": f"tarea numero {numero} de la prueba"}
        cliente.sendall(json.dumps(peticion).encode("utf-8"))
        resultados[numero] = json.loads(cliente.recv(4096).decode("utf-8"))
    except (ConnectionError, OSError, ValueError) as e:
        resultados[numero] = {"status": "error", "message": str(e)}
    finally:
        cliente.close()


def main():
    try:
        cantidad = int(sys.argv[1]) if len(sys.argv) > 1 else 12
    except ValueError:
        print("La cantidad de tareas debe ser un número entero")
        return

    inicio = time.time()
    hilos = [threading.Thread(target=enviar_tarea, args=(n,)) for n in range(1, cantidad + 1)]
    for h in hilos:
        h.start()
    for h in hilos:
        h.join()
    total = time.time() - inicio

    por_worker = {}
    for numero in sorted(resultados):
        r = resultados[numero]
        if r.get("status") == "success":
            worker = r["procesado_por"].split("/")[0]
            por_worker[worker] = por_worker.get(worker, 0) + 1
            print(f"Tarea {numero:>2}: resultado {r['resultado']} -> {r['procesado_por']}")
        else:
            print(f"Tarea {numero:>2}: ERROR {r.get('message')}")

    print(f"\n{cantidad} tareas en {total:.1f} s")
    for worker, n in sorted(por_worker.items()):
        print(f"  {worker}: {n} tareas")


if __name__ == "__main__":
    main()
```

7. Repositorio

Código, diagrama, capturas y README: **https://github.com/AdrianoJere/pfo3-sistema-distribuido-sockets**